

Semestrální práce z předmětu KIV/UPS

Kevin Varchola

17. ledna 2025

Obsah

1	Popis hry	3
1.1	Pravidla hry	3
2	Popis protokolu	3
2.1	Formát zpráv	4
2.2	Sekvence zpráv a jejich význam	4
2.3	Statusové kódy	6
3	Popis implementace	6
3.1	Použité knihovny a verze	6
3.2	Paralelizace	6
4	Struktura projektu	6
4.1	Klient	7
4.2	Server	7
4.3	Další soubory	8
5	Použití a spuštění programu	8
6	Závěr	9

1 Popis hry

”Hra na pistolky” je jednoduchá tahová hra pro dva hráče, kde každý hráč má možnost vybrat z několika akcí (např. shoot, reload, cover). Cílem hry je eliminovat soupeře.

1.1 Pravidla hry

Hra se hraje na tahy:

- Každý hráč má na začátku hry 3 životy a 1 náboj.
- Hráči simultánně vybírají své akce v každém tahu.
- Server po obdržení akcí od obou hráčů vyhodnotí jejich efekt a aktualizuje stav hry.
- Hráč může vybrat akce jako:
 - **Shoot** - vystřelí na soupeře (pokud má náboje).
 - **Reload** - přidá jeden náboj.
 - **Cover** - chrání se proti střelbě.
- Hra končí, když je jeden nebo oba hráči eliminováni.

2 Popis protokolu

Komunikace mezi klientem a serverem je realizována pomocí jednoduchého textového protokolu, který zajišťuje výměnu informací o stavu hry a akcích hráčů. Protokol je navržen tak, aby byl rozšiřitelný a umožňoval snadné přidávání nových typů zpráv.

Celá komunikace začíná inicializační zprávou `hello`, po které následuje sekvence zpráv pro ověření spojení (`ping`), nastavení parametrů (`name`), a poté zprávy související s průběhem hry (`game_ready`, `action`, `round_end`, `game_result`, `game_state`).

2.1 Formát zpráv

Každá zpráva má formát klíč-hodnota oddělený znakem &:

- `request_type`: Typ požadavku odesílaného klientem na server.
- `response_type`: Typ odpovědi vrácené serverem klientovi.
- `status_code`: Kód odpovědi indikující úspěch nebo chybu (např. 200 pro úspěch, 400 pro chybu, 404 pro neznámý požadavek).
- `message`: Volitelná doplňující zpráva nebo popis chyby.
- Další volitelné parametry: Záleží na kontextu (např. `action_type`).

2.2 Sekvence zpráv a jejich význam

1. `hello`: Klient se připojí k serveru a odesílá inicializační zprávu:

- Požadavek:
`request_type=hello`
- Odpověď:
`response_type=hello&status_code=200&message=Hello`

2. `ping`: Testování spojení:

- Požadavek
`request_type=ping`
- Odpověď:
`response_type=ping&status_code=200&message=Pong`

3. `name`: Nastavení jména hráče:

- Požadavek:
`request_type=name&name=Pepa`
- Odpověď:
`response_type=name&status_code=200&message=Player registered`

4. `game_ready`: Oznámení připravenosti ke hře:

- Požadavek:
`request_type=game_ready`
- Odpověď:
`response_type=game_ready&status_code=200&message=Game ready`

5. `player_state`: Získání informací o hráči:

- Požadavek:
`request_type=player_state`
- Odpověď:
`response_type=player_state&status_code=200&message=Health=3, Ammo=1`

6. `action`: Provádění akcí ve hře:

- Požadavek:
`request_type=action&action_type=shoot`
- Odpověď:
`response_type=action&status_code=200&result=Action set`

7. `round_end`: Konec aktuálního kola:

- Požadavek:
`request_type=round_end`
- Odpověď:
`response_type=round_end&status_code=200&message=End`

8. `game_result`: Výsledek hry:

- Požadavek:
`request_type=game_result`
- Odpověď:
`response_type=game_result&status_code=200&message=Win`

9. `game_state`: Celkový stav hry:

- Požadavek:
`request_type=game_state`
- Odpověď:
`response_type=game_state&status_code=200&message=Running`

10. `opponent_state`: Informace o soupeři:

- Požadavek:
`request_type=opponent_state`
- Odpověď:
`response_type=opponent_state&status_code=200&message=Health=3`

2.3 Statusové kódy

- 200:
Požadavek byl úspěšně zpracován.
- 400:
Chyba na straně klienta (např. neplatný formát zprávy, prázdná zpráva, neplatná akce).
- 403:
Zakázaný přístup (např. klient nemá oprávnění vykonat požadovanou akci).
- 404:
Neznámý typ požadavku.
- 408:
Vypršení časového limitu spojení (timeout).
- 409:
Konflikt (např. pokus o registraci hráče s již existujícím jménem).

3 Popis implementace

3.1 Použité knihovny a verze

- Klient: Python 3.11., knihovna `tkinter` pro GUI.
- Server: Go 1.23.

3.2 Paralelizace

Server využívá vlákna pro obsluhu jednotlivých klientů. Každý klient je zpracováván nezávisle, což umožňuje simultánní interakci s více hráči.

4 Struktura projektu

Projekt je rozdělen do dvou hlavních složek: `client/` a `server/`. Každá složka obsahuje specifické soubory pro danou část aplikace.

4.1 Klient

- `client.py`:
Hlavní logika klienta, která spravuje komunikaci se serverem.
- `connect_win.py`:
Modul pro zobrazení okna pro připojení k serveru.
- `const.py`:
Obsahuje konstanty a konfigurační parametry pro klienta.
- `game_win.py`:
Modul implementující hlavní herní okno a interakci uživatele.
- `main.py`:
Spouštěcí soubor klienta.
- `menu_win.py`:
Modul pro hlavní menu aplikace.

4.2 Server

- `main.go`:
Hlavní spouštěcí soubor serveru.
- `game.go`:
Implementace herní logiky na straně serveru.
- `const.go`:
Obsahuje konstanty a nastavení serveru.
- `server.go`:
Modul pro správu komunikace s klienty.
- `structures.go`:
Definuje datové struktury používané v aplikaci.
- `go.mod`:
Konfigurační soubor pro správu závislostí v jazyce Go.

4.3 Další soubory

- **Diagram.pdf:**
Stavový diagram protokolu.
- **Makefile:**
Automatizace spouštění klienta a serveru.
- **Protocol.txt:**
Dokumentace protokolu.

5 Použití a spuštění programu

Program se spouští pomocí **Makefile**, který zjednodušuje proces kompilace a spuštění.

1. Otevřete terminál a přejděte do složky projektu.
2. Ujistěte se, že máte nainstalované všechny potřebné závislosti.
3. Spuštění serveru:
 - Základní příkaz:
`make server` – spustí server na výchozí adrese `0.0.0.0:8080`.
 - S parametry:
`make server ip:port` – umožňuje specifikovat vlastní IP adresu a port (např. `make server 127.0.0.1:9090`).

Po úspěšném spuštění server vypíše zprávu potvrzující, že běží, a naslouchá na zadaném portu.
4. Spuštění klienta:
 - Příkaz: `make client`.
 - Po spuštění klient zobrazí hlavní menu aplikace, odkud se můžete připojit k serveru.

6 Závěr

Téma hry, kterou jsem si vybral, mě velmi bavilo. Během implementace jsem se naučil, jak funguje komunikace mezi serverem a klientem, a to jak v jazyce Go, tak v Pythonu. Projekt mi pomohl lépe pochopit paralelizaci a implementaci komunikačních protokolů. Při práci s paralelizací jsem se však setkal s řadou problémů, přičemž největší výzvou byly deadlocky, které bylo potřeba pečlivě ladit a vyřešit. Co se týče vizuální stránky, GUI by šlo určitě vylepšit, například přidáním animací a obrázků, které jsem původně plánoval implementovat, ale na které nezbyl čas. Celkově projekt přinesl cenné zkušenosti a otevřel nové oblasti ve vývoji.